



CEN 323 - Computer Organization & Assembly Language (COAL)
SEMESTER PROJECT
PHASE 2: FINAL IMPLEMENTATION & DOCUMENTATION

Course Code	CEN 323
Course Title	Computer Organization & Assembly Language (COAL)
Course Instructor	Adnan Jelani
Phase 1 Deadline	3 May 2026 (Proposal COMPLETED)
Phase 2 Deadline	17 May 2026
Demo Schedule	Regular Lab after deadline
Total Marks	100 marks

This document constitutes the official Phase 2 assignment for the CEN 323 Semester Project. Students are expected to implement the project described in their approved Phase 1 proposal, satisfy all Complex Computing Problem (CCA) attributes, and submit all deliverables by the stated deadline.

1 Assignment Overview

Phase 2 is the full implementation phase of the CEN 323 Semester Project. Students must translate their approved proposal into a working 8086 Assembly Language program developed in emu8086. This phase evaluates technical competence, design thinking, documentation quality, teamwork, and the ability to address a Complex Computing Problem.

This assignment is directly mapped to the following Course Learning Outcomes (CLOs):

CLO	Learning Outcome	PLO
CLO1	Identify major components of computer architecture and explain their purposes and interactions.	PLO1: Academic Education
CLO2	Simulate the internal representation of data; show how data is stored and accessed in I/O modules and interconnecting components.	PLO2: Knowledge for Solving Computing
CLO3	Analyze the relationships between hardware architecture and its instruction set.	PLO3: Problem Analysis

2 Complex Computing Problem (CCA) Mapping

This project is mapped to three CCA attributes. These were selected because they are concretely demonstrable within an 8086-assembly project and are not artificially imposed. Students must address each one explicitly in Section 6 of their final report, with specific references to their code.

#	CCA Attribute	What This Means for the Project	CLO	PLO
CCA1	Complex Problem-Solving	The project must address a non-trivial computing problem. A single loop or procedure is insufficient. The solution must involve multi-module interaction, conditional state management, and non-linear program flow that cannot be trivially decomposed.	CLO3	PLO3
CCA4	Design Constraints	Students must work strictly within the 8086 ISA: limited general-purpose registers (AX, BX, CX, DX, SI, DI), 16-bit addressing, segmented memory model, and DOS/BIOS interrupt conventions. No higher-level abstractions are available.	CLO3	PLO3
CCA8	Team Collaboration	Each member must own and implement distinct module(s). Collaboration must be evidenced by meaningful, distributed GitHub commit history from all members throughout the development period not just at submission.	CLO1 CLO2 CLO3	PLO1 PLO2 PLO3

CCA Mapping Note: CCA1 and CCA4 are primarily demonstrated through the code itself. CCA8 is demonstrated through GitHub commit history and the contributions table in the report. Vague statements like 'our project is complex' are not acceptable, cite specific procedures, interrupt calls, or commits.

3 Project Requirements & Technical Specifications

3.1 Development Environment

- Code must be written in 8086 Assembly Language and run in emu8086
- No external libraries other than the built-in one, compiled code, or higher-level language wrappers are permitted.
- The submitted .asm file must be runnable from emu8086 without modification.

3.2 Mandatory Assembly Concept Coverage

Each project must incorporate at least 8 of the 12 lab topics. In the final report, each concept used must be annotated with the corresponding lab number and the procedure/section of code where it appears.

3.3 Complexity Thresholds by Group Size

Projects that fall below the threshold for their group size will receive deductions. The instructor will assess complexity during the live demonstration, and each student will be judged as an individual and as a team member. All members should have 100% understanding of the code and documentation.

4 Deliverables & Submission Instructions

Deliverable	Where to Submit	Deadline
GitHub repository link	Google Sheets (shared link)	3 May 2026
Overleaf project link	Google Sheets (shared link)	
Working .asm source file(s)	GitHub repository (main branch)	17 May 2026
emu8086 project file	GitHub repository	
Zipped Final Report PDF (Generated from LaTeX) + code files	LMS submission	
Live Demonstration	Scheduled lab session	As per timetable

4.1 GitHub Repository Requirements

- Repository must be public, named: CEN323_G<GroupNumber>_<ProjectName> (Group numbers can be seen in the google sheet)
- All group members must have committed code (not documentation alone).
- **README.md** must include project description, setup/run instructions, and a **contributions table** (name, reg. number, module(s) owned).
- **Last-minute bulk commits on deadline day will not be accepted.**

4.2 Final Report Structure (LaTeX/Overleaf)

The report must be written in LaTeX on Overleaf and submitted as a PDF. The collaborative work on the overleaf should reflect the contribution of all team members.

Section	Content Required
1. Cover Page	Bahria Logo, Course, project title, group number, member names, registration numbers, date, Instructor's Name
2. Abstract	100 to 150 words: what the project does, approach used, outcome achieved
3. Introduction	Problem statement, motivation, scope, and objectives
4. System Design	Module/architecture diagram; memory layout; design decisions and their rationale (Use UML standards for the diagrams)
5. Implementation Details	Each procedure/macro described; interrupts used with justification; key algorithmic choices
6. CCA Mapping	One focused paragraph each for CCA1, CCA4, and CCA8 explaining how the project satisfies that attribute with specific code references
7. Testing & Results	Test cases table (input > expected > actual) + emu8086 screenshots of execution
8. Team Contributions	Table: Member Name Registration No. Module(s) Owned GitHub Commits ID
9. Challenges & Lessons Learned	Technical difficulties encountered and how they were resolved
10. Conclusion	Summary of achievements, limitations, and possible future work
11. References	BIOS/DOS references, textbooks, GitHub repository link (IEEE format)

4.3 File Naming Convention

PDF naming format: GroupNumber_EnrollmentNumber_FinalProject.pdf

Example: G04_01-135232-000_FinalProject.pdf

The report should be consistent, and each group member should submit the report according to the naming convention mentioned above.

5 Evaluation Rubric					
Criterion / CCA	Excellent (100%)	Good (75%)	Satisfactory (50%)	Poor (25%)	Marks
A TECHNICAL IMPLEMENTATION (40 marks)					
A1. Functional Correctness <i>CCA1</i>	All features work as designed; edge cases handled; no crashes.	Major features work; minor bugs; minimal edge-case handling.	Partial functionality; some features broken or absent.	Program does not run or core logic fails.	15
A2. Assembly Concept Coverage <i>CCA1, CCA4</i>	8 or more lab concepts correctly applied (see Section 3.2).	6-7 concepts correctly applied with minor misuse.	4-5 concepts; some incorrect usage evident.	3 or fewer concepts; superficial application.	15
A3. Code Modularity & Quality <i>CCA1</i>	Logical use of procedures/macros; meaningful labels; no dead code; readable structure.	Mostly modular; some spaghetti sections; readable.	Partially modular; difficult to follow; minimal structure.	Monolithic code; unreadable; no procedures or macros.	10
B CONSTRAINT ADHERENCE (20 marks)					
B2. Problem Complexity <i>CCA1</i>	Non-trivial; multi-module with state management; meets or exceeds group-size threshold (Section 3.3).	Moderately complex; uses loops, arrays, and procedures together.	Minimal complexity; essentially one module with basic loops.	Trivial project; below complexity threshold for group size.	20
C DOCUMENTATION (15 marks)					
C1. Final Report <i>CCA1, CCA8</i>	Complete, professional LaTeX report; all 11 sections present; CCA mapping section well-argued; matches implementation.	Mostly complete; minor mismatches or missing subsections.	Several sections absent or poorly written; weak CCA discussion.	Report: missing, plagiarized, or does not reflect the implementation.	8
C2. Code Comments & README <i>CCA4</i>	Every procedure and macro commented; README covers setup, features, and roles; clear and accurate.	Major sections commented; README present but incomplete.	Sparse comments; README minimal.	No inline comments; README absent.	7
D COLLABORATION (10 marks)					
D1. GitHub Commit History <i>CCA8</i>	All members have meaningful, distributed commits; history reflects development progression; no bulk push.	All contributed; some members have few commits; mostly meaningful messages.	Uneven contribution; one member dominates the history.	Single-member commits; last-minute bulk push.	6
D2. Role Division <i>CCA8</i>	Each member owns distinct module(s); division documented in README and report; consistent with commits.	Roles documented; minor overlap; mostly evidenced in history.	Roles stated but not clearly divided or evidenced by commits.	No clear division; appears to be one person's work.	4
E DEMONSTRATION (15 marks)					
E1. Live Demo & Q&A <i>CCA1, CCA4, CCA8</i>	Demo runs without issues; all members can explain their own module confidently; technical questions answered.	Minor demo issues; most members explain their parts; most questions answered.	Demo partially functional; some members cannot explain the code.	Demo fails; members cannot explain what the code does.	15

6 Marks Distribution Summary

Sec.	Component	Marks	CCA
A	Technical Implementation	40	CCA1, CCA4
B	Constraint Adherence & Complexity	20	CCA4
C	Documentation	15	CCA1, CCA4, CCA8
D	Collaboration	10	CCA8
E	Demonstration	15	CCA1, CCA4, CCA8
	TOTAL	100	CCA1 CCA4 CCA8

7 Academic Integrity & Policies

7.1 Plagiarism

- Code or documentation copied from the internet, other groups, or generated by AI without disclosure results in a zero for the entire group.
- If AI tools were used at any stage, this must be declared in Section 9 of the report. Undisclosed AI-generated code is treated as plagiarism.
- External references (BIOS and other interrupts documentation, algorithm references) are permitted and encouraged; they must be cited in Section 11.
- Two groups having the same implementation will result in zero for all members.

7.2 Proposal Consistency

- The final implementation must match the approved Phase 1 proposal.
- Scope changes require approval before the final submission. Switching to an entirely different project without approval: deduction of up to 20 marks.

7.3 Late Submission

- 10 marks deducted per day after 17 May 2026 (2 days max, project will be marked zero afterwards)
- Documented justification should be submitted at least 24 hours before the deadline.
- Try to submit days before deadline to save yourself from the hassle.